

SCL — Plan d’upgrade : perception objet, action-accélération, orchestrateur à attention

Etienne Lamy

2026-07-21

Table des matières

0. Diagnostic honnête (pourquoi on repart d’un cran plus bas)	1
1. Fondation — perception OBJET (prédire $T+1$ doit devenir quasi-exact)	2
2. Action = ACCÉLÉRATION (Δv), pas vitesse — et prévoir plusieurs positions	3
3. Arbre de possibilités réel + recherche A^* sur l’état-objet	3
4. Orchestrateur — la STRUCTURE concrète (attention, matrices, gradients, signaux faibles)	4
4.1 Les jetons d’entrée T_t (c’est là que vivent les signaux faibles)	4
4.2 Encodeur — Set Transformer (self-attention multi-tête)	4
4.3 Décodeur — Pointer Network (émettre un programme par POINTAGE)	4
4.4 Comment A^* ENTRAÎNE l’orchestrateur (le point qui manquait)	5
5. Nuit — balayer les possibilités « on aurait pu manger le sucre »	5
6. Viewer — tout loguer, et VOIR l’agent	5
7. Séquencement (chaque étape mesurable, commit + STATUS + tests)	6
8. Ce qu’on garde / jette du code actuel	6

Remise à niveau d’exigence. Ce qui précède n’est pas assez maîtrisé : prévoir $T+1$ est faible, les « branches » sont dégénérées, l’action a été confondue avec la vitesse, et l’entraînement $A^ \rightarrow \text{orchestrateur}$ n’est pas câblé. Ce document dit comment on répare, avec la structure concrète (matrices, gradients, signaux faibles) de l’orchestrateur. 2026-07-21.
Auteur : Etienne Lamy. Fait suite à SCL_conception_action.md.*

0. Diagnostic honnête (pourquoi on repart d’un cran plus bas)

Faiblesse constatée	Preuve mesurée	Cause racine
Prévoir T+1 est médiocre	rappel 57 % (latent opaque) à 84 % (au mieux)	on prédit des pixels bruts , pas des objets ; goulot mal placé
Les « branches » sont dégénérées	(1,0)×10 → (-2,0) figé, identique à (0,0)×10	l'action a été prise comme vitesse soutenue , pas comme Δv ; N3 fragile
Pas d' arbre de possibilités A* n'entraîne pas l'orchestrateur	aucune (jamais construit sur des séquences variées) Mode B n'apprend que sur des programmes de PERCEPTION	manque un état compact où dérouler l'accélération pas de signal A*→politique câblé, pas de signaux faibles en entrée
Réutilisation compositionnelle absente	v=(2,0) non exprimé comme (1,0) (1,0)	pas d'opérateur d'action paramétré ni de recherche qui la trouve
Rien à voir	harnais d'action sans log viewer	log JSONL non émis

Fil conducteur du fix : arrêter de raisonner en **pixels** et raisonner en **objets**. Un champ = un petit ensemble d'objets (**catégorie**, **position**). Dès lors : prédire = **décaler les positions**, l'action = **changer la vitesse**, la recherche = **dérouler des accélérations** sur cet état minuscule, et la compositionnalité ((2,0)=(1,0) (1,0)) devient **native**.

1. Fondation — perception OBJET (prédire T+1 doit devenir quasi-exact)

On a déjà les briques (slot-attention étape 4 : 94 %; VQ catégories étape 5 : 100 % pur). On les assemble en un **état-objet** et on grossit les modèles autant qu'il faut.

Représentation. champ $10 \times 10 \rightarrow E = \{(c_k, p_k, a_k)\}$: pour chaque objet détecté, sa **catégorie** c_k (émergente, VQ), sa **position** $p_k=(x,y)$ relative à l'agent, son **amplitude** a_k . Nombre d'objets petit (~8 visibles). **Compression forte** : $\sim 8 \times (1+2+1)=32$ nombres au lieu de 100 pixels, et surtout **sémantiques**.

Modules par catégorie (tu l'as autorisé). Un **générateur par catégorie** (sucre, bâton, corps) : $g_c(p) \rightarrow$ empreinte de l'objet c à la position p dans le champ. Le champ se **régénère** en sommant les empreintes. Ces générateurs peuvent être **gros** (qualité), seul le **goulot** (la liste E) compte pour la parcimonie (§5). Naissance d'un générateur sur **catégorie nouvelle** (surprise VQ), pas avant.

Prédiction T+1 = décalage (triviale et EXACTE). $E(t+1)$: chaque p_k devient $p_k - v(t)$ (le monde défile à l'opposé du mouvement de l'agent). Régénérer le champ depuis $E(t+1)$. **Cible de rappel** : > **95 %** à T+1, et **stable à T+h** tant que les objets restent visibles (plus d'erreur de décalage cumulée qu'un arrondi entier). *C'est le test qui doit passer avant tout le reste.*

Détecteur de position. Le maillon à muscler : champ $\rightarrow$ positions. Slot-attention (étape 4) donne déjà des slots-objets; on ajoute une **tête de localisation** (argmax/ barycentre softmax par slot) $\rightarrow p_k$. Entraînée par reconstruction (le champ régénéré depuis E doit égaler le champ vu) : gradient **auto-supervisé**, aucune étiquette de position.

Livrable mesurable (étape 23) : rappel $T+1 > 95 \%$, $T+5 > 90 \%$, compression $|E| \approx 100$, catégories émergentes pures. Log viewer : champ VU vs champ RÉGÉNÉRÉ-depuis-E, + les positions d'objets suivies.

2. Action = ACCÉLÉRATION (Δv), pas vitesse — et prévoir plusieurs positions

État dynamique : (E, v) où v = vitesse propre (interne, $\llbracket -2, 2 \rrbracket^2$). **Action** a = accélération $\{(0,0), (\pm 1,0), (0,\pm 1)\}$ = un **changement** de vitesse.

Modèle de transition (compositionnel par construction) :

```
v' = clip(v + a, ±v_max)          # l'accélération met à jour la vitesse (petit module appris)
p_k' = p_k - v'                    # UN opérateur « traduire par v' », réutilisé pour tous l
E' = { (c_k, p_k', a_k) }
```

Deux modules seulement : **accel** : $(v, a) \rightarrow v'$ ($5 \rightarrow 5$, trivial, appris/vérifié étape D) et **translater** : $(E, v') \rightarrow E'$ (décalage). Prédire $T+h$ sous une séquence $a_1 \dots a_h$ = itérer.

Compositionnalité NATIVE (ton test). Aller à $v=(2,0)$ depuis l'arrêt = $a=(1,0)$ **deux fois** : $v:0 \rightarrow 1 \rightarrow 2$, **translater** appliqué avec $v'=1$ puis $v'=2$. Le **même** module **translater** sert à toutes les vitesses ; $v=(2,0)$ **est** $(1,0)$ enchaîné — il n'existe pas de « module $(2,0)$ » séparé, il **émerge** de la double application. C'est exactement « se rendre compte que $v=(2,0)$ se simule par double usage du module $(1,0)$ ». **Critère d'acceptation §31** : la recherche/l'orchestrateur DOIT trouver que **translater translater** prédit l'outcome de $(2,0)$ — mesuré, pas supposé.

Livrable (étape 24) : matrice de prédiction position sous **séquences d'actions variées** (plus de $(1,0) \times 10$ figé) ; vérifier que $(1,0)$ appliqué $2 \times$ = outcome réel de vitesse 2.

3. Arbre de possibilités réel + recherche A^* sur l'état-objet

Sur (E, v) — minuscule — dérouler l'arbre des accélérations est **bon marché**.

- **Nœuds** = (E, v) prédits ; **arêtes** = accélérations a (coût = temps/effort).
- **But** = un sucre atteint (un p_k de catégorie sucre arrive sur le corps).
- $g(n)$ = coût cumulé ; $h(n)$ = distance-valeur au sucre le plus proche **apprise** (**recherche.ValeurApprise**, TD) — au début 0 (Dijkstra borné, §7.3), puis informative.
- $A^*(\text{recherche.a_etoile, déjà écrit})$ sur **voisins** = $\{(\text{translater}(\text{accel}(v, a)), a)\}$. Budget borné on **ouvre puis abandonne** des branches, on pousse **une** action.

Comme on prévoit en **objets**, « voir » le sucre à 3-4 actions et **prioriser** cette branche devient direct : l'arbre trouve la séquence qui amène un p_sucre sur le corps, h la remonte. **La nuit**, budget large : on **balaie** les séquences depuis les épisodes stockés et on **s'aperçoit** qu'une autre suite d'accélérations aurait mangé le sucre $\rightarrow$ cible d'entraînement (valeur + politique). *Mais tout cela suppose la fondation §1 : sans voir/prévoir le sucre, rien n'est possible — d'où la priorité absolue à §1.*

Livrable (étape 25) : depuis un état où un sucre est à 3-4 actions, A^* **prioritise** la branche qui le mange (mesuré : longueur/qualité du plan, sucre atteint $>$ hasard) ; branches **non dégénérées** (des séquences différentes donnent des futurs différents).

4. Orchestrateur — la STRUCTURE concrète (attention, matrices, gradients, signaux faibles)

L'orchestrateur **ne calcule pas**, il **compose** : il lit un **ensemble de jetons** (un par module actif + jetons spéciaux) et **émet un programme** (suite de triplets). Il est déjà à ~80 % dans `attention.py` (Set Transformer + Pointer Network + REINFORCE) ; on ajoute les **signaux faibles en entrée** et le **signal A* en sortie**.

4.1 Les jetons d'entrée T_t (c'est là que vivent les signaux faibles)

Un jeton par module actif i , vecteur $x_i \in \mathbb{R}^d$ ($d=64$) :

$x_i = \text{LayerNorm}(W_{\text{lat}} \cdot z_i + (\text{type}_i) + W_{\text{sig}} \cdot s_i)$

- z_i : dernier **output/latent** du module (dim propre $\rightarrow$ projeté par $W_{\text{lat}} \in \mathbb{R}^{d \times \cdot}$) ;
- (type_i) : **embedding de type** appris (champ / latent / état / action) $\in \mathbb{R}^d$;
- s_i : **SIGNAUX FAIBLES** (ce que tu demandes), vecteur concaténé puis projeté par $W_{\text{sig}} \in \mathbb{R}^{d \times m}$: $s_i = [\text{activation}_i, \text{récence}_i, \text{condensateur_reco}_i, \text{condensateur_gen}_i, \text{reco}_i, \text{gen}_i, \text{incertitude}_i, \text{progrès}_i]$ (activation/récence = ce module a-t-il servi récemment ; condensateurs = ses statistiques accumulées ; = fiabilités prévision/génération $[0,1]$; incertitude/progrès = qualité d'apprentissage). **La politique apprend à pointer selon ces signaux** (ex. « module peu fiable ici $\rightarrow$ ne pas l'utiliser »).
- Jetons spéciaux ajoutés : **[NEW]** (créer un module), **objectif** (embedding du besoin dominant, §17), **trace_{t-1}** (le programme émis au pas précédent — déroulé continu).

Matrice d'entrée : $X \in \mathbb{R}^{N \times d}$ ($N = \# \text{modules actifs} + \text{spéciaux}$). **Aucun encodage positionnel** (invariance par permutation — Set Transformer).

4.2 Encodeur — Set Transformer (self-attention multi-tête)

Par couche (h têtes, $d_h = d/h$) :

```
Q = X W_Q ,   K = X W_K ,   V = X W_V           # W_Q, W_K, W_V ∈ ℝ^{d×d}
A = softmax( Q K^T / √d_h )                     # A ∈ ℝ^{N×N}, poids d'attention entre modules
H = LayerNorm( X + (A V) W_O )                   # W_O ∈ ℝ^{d×d}
H = LayerNorm( H + FFN(H) )                      # FFN : d→4d→d
```

Sortie $H \in \mathbb{R}^{N \times d}$: chaque module **contextualisé par les autres** (l'attention A dit quels modules comptent l'un pour l'autre). Empilable en L couches.

4.3 Décodeur — Pointer Network (émettre un programme par POINTAGE)

État de décodage $q_t \in \mathbb{R}^d$ (GRU sur les triplets déjà émis + H agrégé). À chaque pas on émet un **triplet** (source, opérateur, cible) en **pointant** des indices de H (jamais un vocabulaire fixe) :

```
p_src(j) = softmax_j( u_src tanh( W_src [H_j ; q_t] ) )   # j sur les N jetons
p_op      = softmax( E_op q_t )                           # E_op : table d'opérateurs appris
p_cib(j)  = softmax_j( u_cib tanh( W_cib [H_j ; q_t] ) )   masque_type(op) # -∞ si type inco
```

opérateurs incluent désormais **agir(a)**, **translater**, **accél**, **compresser**, **generer**, **predire**, **id**, **[EOF]**. Émission jusqu'à **[EOF]**. Le **masquage de type** rend les triplets absurdes impossibles (déjà **attention.masque_compatibilite_type**).

4.4 Comment A* ENTRAÎNE l'orchestrateur (le point qui manquait)

A* (Mode A, **recherche.a_etoile**) est le **professeur** : sur l'état courant il **cherche** le meilleur programme **P*** et sa **valeur** $R^* = G - \gamma \cdot \text{coût}$ (ou : atteint le but / valeur $g+h$). On entraîne l'émetteur (Mode B) par **deux gradients**, $\text{loss} = L_{\text{imit}} + \gamma \cdot L_{\text{rl}} - \gamma \cdot H$:

1. **Imitation** (démarrage à froid) : $L_{\text{imit}} = - \sum_t \log p(\text{triplet}_t = P^*_t)$ (teacher forcing sur les triplets de **P***). Reproduit la recherche **sans la refaire**.
2. **Renforcement** (dépasse le prof) : échantillonner $P \sim \pi$, l'exécuter, mesurer R ; $L_{\text{rl}} = - (R - b) \sum_t \log p(\text{triplet}_t)$, baseline $b = \text{regret de composition}$ (**credit.regret_composition**, déjà écrit) ou moyenne mobile.
3. **Entropie** H recuite (anti-effondrement, éprouvé étape 14).

Gradients (locaux à l'orchestrateur). loss/γ remonte : têtes de pointage ($u_{\text{src}}, W_{\text{src}}, E_{\text{op}}, u_{\text{cib}}, W_{\text{cib}}$) → GRU décodeur → encodeur ($W_Q, W_K, W_V, W_O, \text{FFN}$) → projections d'entrée ($W_{\text{lat}}, \dots, W_{\text{sig}}$). **Adam**, $\text{lr } 1e-3$, clip-norm (anti-divergence des pointeurs, déjà en place). **Aucun gradient n'entre dans les modules** : ils apprennent **localement** ; l'orchestrateur n'apprend qu'à les **composer** en lisant leurs signaux faibles. C'est la séparation §10.8 ($\bar{g}_{\text{orch}}$ distinct de $\bar{g}_i$, **attention.AccumulateurOrchestrateur**).

Ce que ça donne concrètement pour ta compositionnalité : parmi les programmes qu'A* évalue figure **translater translater** ; s'il prédit l'outcome de $v=(2,0)$ mieux (ou aussi bien pour moins cher) qu'un module dédié, **A* le classe premier**, l'imitation l'apprend, et l'orchestrateur émet **(1,0) deux fois** au lieu d'inventer un module $(2,0)$. La réutilisation émerge **par la mesure**.

Livable (étape 26) : sur un état donné, l'orchestrateur émet le programme qu'A* juge meilleur (imitation) puis l'améliore (REINFORCE) ; démontrer $(2,0) = (1,0) (1,0)$ **choisi** par l'orchestrateur ; ablation montrant que **retirer les signaux faibles dégrade** le choix (preuve qu'ils servent).

5. Nuit — balayer les possibilités « on aurait pu manger le sucre »

Rejeu des épisodes (mémoire, étape 12/20) **en objets** : depuis $E(t_0)$ d'un épisode, dérouler plusieurs séquences d'accélération (A* budget large) et **détecter** celles qui amènent un sucre sur le corps → cibles positives pour **h (valeur)** et pour la **politique** (l'orchestrateur apprend à émettre ces séquences). C'est le « de plus en plus amont » de l'étape 20, mais **rendu possible** parce qu'en objets on **voit et prédit** le sucre.

6. Viewer — tout loguer, et VOIR l'agent

- **Log partout** : chaque harnais action/horizon prend **--log** et émet le vocabulaire viewer.
- **Nouveau panneau « agent »** : champ VU + objets suivis (catégorie, position) + l'arbre A* en cours (branches ouvertes/abandonnées) + l'action poussée + les pulsions. On **voit** l'agent viser le sucre et délibérer.

- **Panneau orchestrateur** : la matrice d'attention **A** (quels modules s'écoutent), les signaux faibles par module, le programme émis.

7. Séquencement (chaque étape mesurable, commit + STATUS + tests)

#	Livrable	Critère de réussite	Dépend de
23	Perception objet + prédiction par décalage	rappel T+1 > 95 % , T+5 > 90 %, E 100, catégories pures	slot(4)+VQ(5)
24	Action = accélération , transition (E, v, a) → (E', v')	prédiction sous séquences variées ; (1,0)²=outcome (2,0)	23
25	Arbre A* sur l'état-objet	depuis 3-4 actions, prioritise la branche-sucre ; branches non dégénérées	24
26	Orchestrateur à attention + signaux faibles, entraîné par A*	émet le programme d'A* (imit.+RL) ; choisit (1,0) (1,0) ; ablation signaux faibles	25, attention.py
27	Nuit : balayage → « on aurait pu manger »	la visée du sucre s'améliore de nuit en nuit (monde frais gelé)	25, 26
28	Viewer agent + arbre + attention	on voit l'agent viser, l'arbre s'ouvrir, la matrice d'attention	tous

Priorité absolue : étape 23. Tant que « voir et prévoir T+1 » n'est pas quasi-exact, tout le reste est bâti sur du sable — c'est ton point, et il commande l'ordre.

8. Ce qu'on garde / jette du code actuel

- **On garde** : module_attention.py (slots), classification_emergente.py (VQ), attention.py (Set Transformer + Pointer + REINFORCE — on l'active enfin pour l'action), recherche.py (A* + ValeurApprise), curiosite.py, pulsions.py, credit.py, nuit_*
- **On refond** : action.py/planification.py (Q sur pixels bruts → **valeur/politique sur état-objet** ; action = **accélération**) ; regime.py reste utile comme détecteur mais n'est plus la voie de prédiction du champ.

- **On abandonne** comme voie principale : la prédiction **pixel-brut** multi-pas (gardée seulement comme **ancree de vérité** ponctuelle, §7.4).